

underlying layers with better reliability.

## Response codes

Most response codes are similar to HTTP. For instance, 2.xx indicates success, 4.xx indicates client error and 5.xx indicates server error. Here are some frequently encountered response codes:

|      |                      |
|------|----------------------|
| 2.01 | Created              |
| 2.02 | Deleted              |
| 2.04 | Changed              |
| 2.05 | Content              |
| 4.04 | Not found (resource) |
| 4.05 | Method not allowed   |

This is the summary of the CoAP protocol and message format. You can refer to the listed RFC and references for more details like message ID rules, token generation, options, etc. Now, let's go ahead with some hands-on stuff. The Eclipse IoT project offers Californium core library, demo apps and add-ons to bootstrap with the protocol.

## Building the Eclipse Californium core demo apps

Set up the Maven build using the recent stable version of binaries. Check out the Git repository [github.com/eclipse/californium.git](https://github.com/eclipse/californium.git) with a recent stable branch, say 1.0.4, and switch into a cloned directory, as follows:

```
mvn clean install
# In case of failure at scandium-core module comment the
following line in pom.xml under modules as follows
<!-- <module>scandium-core</module> -->

# Or rerun the command as follows
mvn clean installed -rf :californium-core

# collect californium-core-1.0.4.jar from californium-
core/target and element-connector-1.0.4.jar from element-
connector/target to a new work directory for building your
own server, client apps without maven build support.
```

*element-connector* is a Java socket abstraction for UDP, DTLS, TCP, etc., which provides a connector interface implemented by UDPConnector currently, and DTLS by Scandium plugin. Support for TCP, SMS can be added in future with this abstraction layer.

## A tour of demo apps and quick test with the Firefox Copper plugin

Switch to demo apps/run and launch the following demos as follows:

```
java -jar cf-helloworld-server-1.0.4.jar
```

This provides a single resource at path */helloWorld* supporting the GET method.

Open Firefox, install the Copper plugin using <https://addons.mozilla.org/en-US/firefox/addon/copper-270430/>. Type the URL *coap://localhost*, and hit *Ping* to check how alive the server is and hit *Discover* to retrieve hosted resources. Click on the *HelloWorld* resource and request the GET method, which returns the response with the payload *Hello World!* You will also notice that other methods are not allowed, which is indicated by the response code 4.05.

You can run another demo, which provides various resources supporting REST methods and also the *Observe* method and block wise transfer, as follows:

```
java -jar cf-plugtest-server-1.0.4.jar
```

You can also run *cf-helloworld-client-1.0.4.jar* and *cf-plugtest-client-1.0.4.jar* for testing the respective servers.

## Working with CoAP servers with custom resources

To run a server with custom resources, you can use the example code at [github.com/rajeshsola/iot-examples/coap/demo-server](https://github.com/rajeshsola/iot-examples/coap/demo-server), which defines the following resources with supported operations. In this demo, the resources represent dummy information in order to run without specific hardware.

|              |                              |
|--------------|------------------------------|
| /Temperature | GET                          |
| /RtcData     | Observe, GET                 |
| /Leds        | subpath for red, green, blue |
| /Leds/xxx    | PUT, GET                     |
| updateme!    | PUT, POST, GET               |
| removeMe!    | DELETE                       |

Execute the *make* command with *build*, *run* targets to build and run the demo server. You can test this server using the Firefox Copper plugin as shown in Figure 3.

## NodeRED and CoAP-cli support

NodeRED is a visual wiring tool for prototyping IoT solutions and networking services. It comes with an add-on *node-red-contrib-coap* and provides a *coap-request* node for making

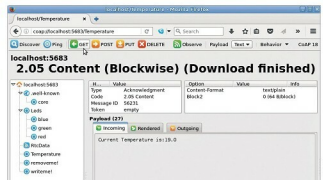


Figure 3: Firefox Copper plugin